

Competitive Programming

Section 39 Editorial: Heaps and Priority Queues

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
2	A. Running Median	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. Course Rooms	6
3.1	Restatement	6
3.2	Algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	7
4	C. K Smallest Pair Sums	8
4.1	Restatement	8
4.2	Algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	9
5	Testing discussion	10
6	Detailed upsolve workshop	11
6.1	A. Running Median	11
6.2	B. Course Rooms	11
6.3	C. K Smallest Pair Sums	12
7	D. Merge Costs	13
7.1	Restatement and traps	13
7.2	Observation and trace	13
7.3	Algorithm	13
7.4	Correctness proof	13
7.5	Complexity	14
7.6	Full C++ solution	14
7.7	Full Python solution	15
7.8	Implementation notes and tests	15
7.9	Stress testing	15
8	What to carry forward	16

1 Editorial overview

Problem	Main idea
A. Running Median	See full algorithm and code below.
B. Course Rooms	See full algorithm and code below.
C. K Smallest Pair Sums	See full algorithm and code below.

2 A. Running Median

2.1 Restatement

After every inserted number, print the lower median of the prefix.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
```

```

if (!(cin >> n)) return 0;
priority_queue<long long> low;
priority_queue<long long, vector<long long>, greater<long long>> high;
for (int i = 0; i < n; i++) {
    long long x;
    cin >> x;
    if (low.empty() || x <= low.top()) low.push(x);
    else high.push(x);
    if (low.size() < high.size()) {
        low.push(high.top());
        high.pop();
    }
    if (low.size() > high.size() + 1) {
        high.push(low.top());
        low.pop();
    }
    if (i) cout << ' ';
    cout << low.top();
}
cout << '\n';
return 0;
}

```

2.6 Full Python solution

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    low, high = [], []
    out = []
    for x in data[1:1 + n]:
        if not low or x <= -low[0]:
            heapq.heappush(low, -x)
        else:
            heapq.heappush(high, x)
        if len(low) < len(high):
            heapq.heappush(low, -heapq.heappop(high))
        if len(low) > len(high) + 1:
            heapq.heappush(high, -heapq.heappop(low))
        out.append(str(-low[0]))
    print(" ".join(out))

if __name__ == "__main__":
    main()

```

3 B. Course Rooms

3.1 Restatement

Given half-open intervals $[l, r)$, print the minimum number of rooms needed.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
```

```

if (!(cin >> n)) return 0;
vector<pair<int,int>> intervals(n);
for (auto &[l, r] : intervals) cin >> l >> r;
sort(intervals.begin(), intervals.end());
priority_queue<int, vector<int>, greater<int>> pq;
int ans = 0;
for (auto [l, r] : intervals) {
    while (!pq.empty() && pq.top() <= l) pq.pop();
    pq.push(r);
    ans = max(ans, (int)pq.size());
}
cout << ans << '\n';
return 0;
}

```

3.6 Full Python solution

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    intervals = [(data[i], data[i + 1]) for i in range(1, 2 * n + 1, 2)]
    intervals.sort()
    heap = []
    ans = 0
    for l, r in intervals:
        while heap and heap[0] <= l:
            heapq.heappop(heap)
        heapq.heappush(heap, r)
        ans = max(ans, len(heap))
    print(ans)

if __name__ == "__main__":
    main()

```

4 C. K Smallest Pair Sums

4.1 Restatement

Two arrays are sorted. Print the k smallest values of $a[i]+b[j]$.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
```

```

if (!(cin >> n >> m >> k)) return 0;
vector<long long> a(n), b(m);
for (auto &x : a) cin >> x;
for (auto &x : b) cin >> x;
using State = tuple<long long,int,int>;
priority_queue<State, vector<State>, greater<State>> pq;
for (int i = 0; i < n; i++) pq.emplace(a[i] + b[0], i, 0);
for (int t = 0; t < k; t++) {
    auto [s, i, j] = pq.top();
    pq.pop();
    if (t) cout << ' ';
    cout << s;
    if (j + 1 < m) pq.emplace(a[i] + b[j + 1], i, j + 1);
}
cout << '\n';
return 0;
}

```

4.6 Full Python solution

```

import heapq
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m, k = data[0], data[1], data[2]
    a = data[3:3 + n]
    b = data[3 + n:3 + n + m]
    heap = [(a[i] + b[0], i, 0) for i in range(n)]
    heapq.heapify(heap)
    out = []
    for _ in range(k):
        s, i, j = heapq.heappop(heap)
        out.append(str(s))
        if j + 1 < m:
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    print(" ".join(out))

if __name__ == "__main__":
    main()

```

5 Testing discussion

6 Detailed upsolve workshop

Read the following cards before treating the reference listings as templates. Each one states the exact heap meaning; that meaning is the proof and the debugging oracle.

6.1 A. Running Median

Constraints and diagnosis. Up to $2e5$ numbers arrive online, so sorting every prefix is quadratic. Print the lower median: for an even prefix this is the smaller of the two middle values.

Trace and pseudocode. For 5,1,9,2, insert into `low` when it belongs to the lower half, otherwise into `high`; move the top element until `len(low)==len(high)` or `len(low)==len(high)+1`. Outputs are 5,1,5,2.

```
insert x into the appropriate heap
if low is smaller: move high.top to low
if low has two more elements: move low.top to high
print low.top
```

Proof and complexity. The partition invariant says every low value is at most every high value, while the size invariant makes `low.top` the element at rank $(len+1)/2$. Insertion and one rebalance cost $O(\log n)$, with $O(n)$ memory. In C++ use a max-heap plus `greater<long long>` min-heap; in Python negate `low` values. Test duplicates, negatives, descending input, and every prefix length. A slow oracle sorts each prefix.

6.2 B. Course Rooms

Constraints and diagnosis. With up to $2e5$ half-open intervals, test all pairs is quadratic. Sort by start; the min-heap is precisely the end times of currently active courses.

Trace and pseudocode. For $[0,10)$, $[5,7)$, $[6,20)$, $[10,12)$, the heap sizes after push are 1,2,3,2; answer three.

```
sort intervals by start
for [l,r):
    while heap.top <= l: pop
    push r
    answer = max(answer, heap.size)
```

Proof and complexity. After cleanup, a heap end is greater than `l` iff its interval overlaps the current one. Thus its size is the active overlap count, and the maximum is the minimum number of rooms. Sorting costs $O(n \log n)$ and heap work is $O(n \log n)$. The strictness test is `<=`, not `<`; test touching intervals, nested intervals, and equal start times.

6.3 C. K Smallest Pair Sums

Constraints and diagnosis. Arrays are sorted, and k is far smaller than $n*m$ in useful cases. Materializing all pairs loses both time and memory. Each fixed- a row is sorted, so this is a k -way merge.

Trace and pseudocode. For $a=[1,4,8]$, $b=[2,3,10]$, seed $(3,0,0)$, $(6,1,0)$, $(10,2,0)$. Popping then advancing its row emits 3,4,6,7,10 for the first five sums.

```
push (a[i]+b[0], i, 0) for each seeded row
repeat k times:
    pop (sum,i,j), output sum
    if j+1<m: push (a[i]+b[j+1],i,j+1)
```

Proof and complexity. The heap contains the first unreported item of every row. Every later item in a row is no smaller, so the heap minimum is the global next sum. Time is $O((n+k) \log n)$ with the local all-row seed (or $O((\min(n,k)+k) \log \min(n,k))$ when optimized); memory is the heap size. Use `long long` sums, tuples in both languages, and test ties, $k=1$, one row, and negative values against an all-pairs sorting oracle.

7 D. Merge Costs

7.1 Restatement and traps

Repeatedly merge two current files, paying their combined size, until one file remains. Minimize total cost. With $n \leq 200000$, trying merge orders is exponential; totals require `long long`. A one-file input costs zero.

7.2 Observation and trace

Always merge the two smallest current sizes. For 4,3,2,6, merge $2+3=5$, then $4+5=9$, then $6+9=15$, totaling 29.

View any merge plan as a binary tree whose leaves are original files. A file size is charged once for every ancestor merge, so smaller sizes should be at least as deep as larger sizes. Thus some optimal tree makes the two smallest files siblings at maximum depth; merge them first and recurse on their sum.

7.3 Algorithm

```
put every size in a min-heap
while heap has at least two values:
    pop a and b
    answer += a+b
    push a+b
print answer
```

7.4 Correctness proof

Lemma. There is an optimal merge plan whose first merge uses two smallest current sizes.

Proof. In an optimal merge tree, choose two sibling leaves at greatest depth. Swapping either of their leaf sizes with a no-larger smallest size can only decrease the weighted depth sum, so the two smallest sizes can occupy those sibling leaves without worsening the plan. Contracting that sibling pair is exactly merging them first. $\square$

Theorem. The heap algorithm has minimum total cost.

Proof. By the lemma, its first merge is compatible with an optimal plan. After replacing the pair by their sum, the remaining problem has the same form. Induction on the number of files proves every greedy merge and therefore the total is optimal. $\square$

7.5 Complexity

$n-1$ heap merges cost $O(n \log n)$ time and $O(n)$ memory.

7.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n))
        return 0;

    priority_queue<
        long long,
        vector<long long>,
        greater<long long>
    > heap;

    for (int i = 0; i < n; ++i) {
        long long size;
        cin >> size;
        heap.push(size);
    }

    long long answer = 0;
    while (heap.size() > 1) {
        long long first = heap.top();
        heap.pop();

        long long second = heap.top();
        heap.pop();

        long long merged = first + second;
        answer += merged;
        heap.push(merged);
    }

    cout << answer << '\n';
    return 0;
}
```

7.7 Full Python solution

```
import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

    sizes = data[1:]
    heapq.heapify(sizes)

    answer = 0
    while len(sizes) > 1:
        first = heapq.heappop(sizes)
        second = heapq.heappop(sizes)
        merged = first + second

        answer += merged
        heapq.heappush(sizes, merged)

    print(answer)

if __name__ == "__main__":
    main()
```

7.8 Implementation notes and tests

Use a min-heap (`greater<long long>` in C++, `heapq` in Python). Do not sort once: merged sums must re-enter the current frontier. Test one file, equal sizes, a very large size beside small ones, and totals beyond 32-bit range.

7.9 Stress testing

For at most seven files, recursively try every pair of current files and take the least cost; this is independent of the greedy rule. The scale generator uses 200000 ones and a heap-computed expected total to exercise heap work and 64-bit accumulation.

8 What to carry forward

A heap represents the next useful frontier, not an arbitrary bag of values. State exactly what each heap item means, prove that its top is globally next, and check the boundary that removes or advances a frontier item.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.